Student Notes: Databases & Memory for AI Agents

Workshop Overview

Duration	90 minutes
Goal	Build a database that gives your AI agent memory
Tools	ChatGPT Enterprise + Supabase (browser only)
Outcome	Working database with tables, security policies, and search

---

What You'll Build

BEFORE: Stateless Agent

- User: "I'm vegetarian"
- Agent: "OK, noted!"
- *(next day)*
- User: "What should I eat?"
- Agent: "How about a nice steak?" ❌

AFTER: Agent with Memory

- User: "I'm vegetarian"
- Agent: "OK, noted!" → *saves to database*
- *(next day)*
- User: "What should I eat?"
- Agent: *searches database* → "You mentioned you're vegetarian. How about risotto?" ✅

Tools You Need (All Browser-Based)

Tool	URL	Purpose
ChatGPT Enterprise	chat.openai.com	Generate SQL, debug errors
Supabase	supabase.com	Database hosting, SQL editor

No terminal/CLI needed! Everything happens in your browser.

Quick Start Checklist

- [ ] ChatGPT Enterprise open in one tab
- [ ] Supabase account created
- [ ] New Supabase project (region: Frankfurt)
- [ ] SQL Editor found (left sidebar)

Phase 1: Homework Validation (3 min)

Copy this prompt into ChatGPT:

```
You are my mentor for code security audit. Yesterday I was assigned to analyze
the GitHub repo ai-academy-dashboard and find security bugs.

Your task:
1. Ask me 5 questions about my work (one at a time)
2. After each answer, give me brief feedback
3. At the end, give me an overall rating (1-5 stars)

Questions should cover:
- What tool/prompt did I use for the analysis
- What categories of bugs did I look for (secrets, injection, RLS...)
- Specific findings and their severity
- Proposed solutions
- What I learned about security

If I didn't do the task, help me catch up quickly in 3 minutes.

Start with the first question.
```

Phase 2: Schema Design

Step 1: Ask ChatGPT to generate SQL

```
Design a database schema for Supabase (PostgreSQL) for an AI agent with memory.

I need tables:
1. messages - conversation history (user_id, role, content, created_at)
2. user_facts - facts about the user (user_id, fact_type, fact_value, confidence)
```

```
3. sessions - session management (user_id, started_at, ended_at)

Requirements:
- UUID for all IDs
- user_id references auth.users
- Timestamps with timezone
- Indexes for common queries
- JSONB for metadata where it makes sense

Generate complete SQL for Supabase.
```

Step 2: Run in Supabase

1. Copy the SQL from ChatGPT
2. Open Supabase → SQL Editor
3. Paste and click **Run**
4. If error → copy error → paste back to ChatGPT → fix → repeat

✅ Checkpoint

Open **Table Editor** (left sidebar). You should see:

- `messages` table
- `user_facts` table
- `sessions` table

Phase 3: Row Level Security (RLS)

Why RLS?

Without RLS, anyone with your API key can read ALL data. With RLS, the database itself enforces "users can only see their own data."

Step 1: Ask ChatGPT for policies

```
I need RLS (Row Level Security) policies for Supabase.

My tables:
- messages (id, user_id, role, content, created_at)
- user_facts (id, user_id, fact_type, fact_value)
- sessions (id, user_id, started_at, ended_at)

Rules:
1. Users can only see and modify their OWN records
```

2. user_id must equal auth.uid()
3. INSERT must have user_id = auth.uid()
4. Admin role can see everything

Generate:
- ALTER TABLE ... ENABLE ROW LEVEL SECURITY for each table
- CREATE POLICY for SELECT, INSERT, UPDATE, DELETE
- Comments explaining each policy

Step 2: Run in Supabase

Same process: Copy → SQL Editor → Run

Step 3: Test Your Security (Red Team!)

Try these queries in SQL Editor:

```sql
-- Should return ONLY your messages
SELECT * FROM messages;

-- Should FAIL (wrong user_id)
INSERT INTO messages (user_id, role, content)
VALUES ('00000000-0000-0000-0000-000000000000', 'user', 'hack attempt');

-- Should return 0 rows
SELECT * FROM user_facts WHERE user_id != auth.uid();
```

✅ Checkpoint

Go to **Authentication → Policies**. You should see policies for all 3 tables.

Phase 4: Search Function

Option A: Use Supabase AI (Easier)

1. In SQL Editor, click the ✨ button
2. Type: "Create a function to search messages by keyword for a specific user"
3. Review the generated SQL
4. Click Run

Option B: Use ChatGPT (More Control)

Create a PostgreSQL function for Supabase:

Name: search_user_messages
Parameters:
- search_query (text)
- user_uuid (uuid)
- max_results (int, default 10)

The function should:
1. Search in messages.content using full-text search
2. Return only messages for the given user (RLS compliance)
3. Order by relevance
4. Limit to max_results

Bonus: If I have pgvector extension, add similarity search too.

Test Your Function

```sql
-- Add test data first
INSERT INTO messages (user_id, role, content) VALUES
  (auth.uid(), 'user', 'I am vegetarian and love Italian cuisine'),
  (auth.uid(), 'assistant', 'Great! I recommend trying risotto ai funghi.'),
  (auth.uid(), 'user', 'I have a nut allergy');

-- Test the search
SELECT * FROM search_user_messages('vegetarian', auth.uid(), 5);
```

✅ Checkpoint

Your function returns results filtered by user_id.

What You Built

Component	Purpose
messages table	Stores conversation history
user_facts table	Stores extracted preferences
sessions table	Tracks conversation sessions
RLS policies	Ensures data isolation per user
search function	Finds relevant memories

How It Connects to Your Agent

```typescript
import { createClient } from '@supabase/supabase-js'

const supabase = createClient(SUPABASE_URL, SUPABASE_ANON_KEY)

// Before responding - load relevant history
async function getRelevantMemories(question: string, userId: string) {
  const { data } = await supabase.rpc('search_user_messages', {
    search_query: question,
    user_uuid: userId,
    max_results: 5
  })
  return data
}

// After responding - save the interaction
async function saveMessage(userId: string, role: string, content: string) {
  await supabase.from('messages').insert({ user_id: userId, role, content })
}
```

Homework: Semantic Search with pgvector

Today we built **keyword search**. Homework is **semantic search** (finds meaning, not just words).

Steps

1. **Enable pgvector extension**
   - Supabase Dashboard → Database → Extensions
   - Search for "vector" → Enable
2. **Add embedding column**

```sql
ALTER TABLE messages
ADD COLUMN embedding vector(1536);
```

3. **Create similarity search function**
   - Ask ChatGPT: "How to implement semantic search in Supabase with pgvector step by step"

4. **Bonus: Edge Function for embeddings**
    - Creates embeddings automatically when messages are inserted

---

Troubleshooting

"ChatGPT generated wrong SQL"

Copy the error message back to ChatGPT:

```
I got this error when running your SQL:
[paste error here]

Please fix it.
```

"RLS is blocking everything"

Check if you have policies for the operation you're trying:

```sql
-- See all your policies
SELECT * FROM pg_policies WHERE tablename = 'messages';
```

"I can't find SQL Editor"

Left sidebar → Look for database icon → SQL Editor

"Supabase AI doesn't understand me"

Be more specific. Instead of:

> "Make a search"

Try:

> "Create a PostgreSQL function called search_messages that takes a text parameter and searches the content column of the messages table"

Key Concepts Summary

Concept	One-Liner
RLS	Database-level security that apps can't bypass
pgvector	PostgreSQL extension for storing/searching vectors

Concept	One-Liner
Embedding	Text converted to numbers that capture meaning
Full-text search	Keyword matching with stemming and ranking
Semantic search	Meaning-based search using vector similarity

Useful Links

Resource	URL
Supabase Docs	docs.supabase.com
pgvector Guide	supabase.com/docs/guides/ai
RLS Docs	supabase.com/docs/guides/auth/row-level-security
Workshop Page	[provided by mentor]

Notes Space

Use this area for your own notes during the workshop: